MULTIMODAL PDF INTELLIGENCE LAB

RAG, OCR ve Görsel Doküman Anlama Üzerine Üç Aşamalı Geliştirme Use Case'i

Hedef Katılımcı
Gazi Üniversitesi Bilgisayar Mühendisliği 2. sınıf öğrencisi Fatih. Use case; temelden başlayarak RAG mantığını gerçekten anlamasını, OCR ve document-AI bileşenlerini uygulamasını ve sonunda GitHub/LinkedIn'de sergilenebilecek portföy kalitesinde bir proje ortaya çıkarmasını amaçlar.

Hazırlanma tarihi: Ekim 2026

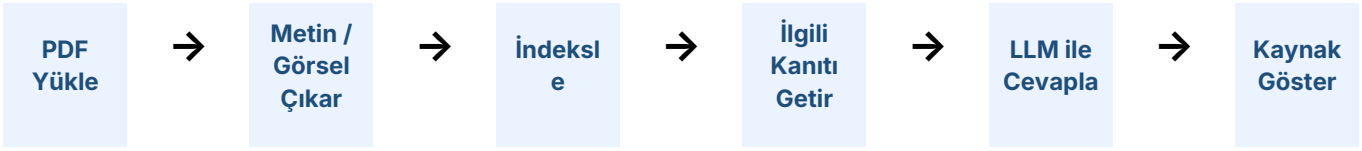
Seviye: Başlangıç → Orta → Portföy/Bitirme Projesi

İçindekiler

1. Use Case'in Amacı ve Öğrenme Çıktıları 3
2. RAG ve OCR'ı Doğru Konumlandırma 3
3. Aşama 1 - PDF Tabanlı Temel RAG Web Uygulaması 4
4. Aşama 2 - OCR ve Multimodal Doküman Anlama 6
5. Aşama 3 - Final Projesi: HybridDoc AI 8
6. Önerilen Teknik Mimari ve Teknoloji Yığını 10
7. Test, Ölçüm ve Başarı Kriterleri 11
8. GitHub ve LinkedIn Portföy Standardı 12
9. Geliştirme Takvimi ve Teslimat Kontrol Listesi 13
10. Kaynaklar ve Güncel Teknoloji Notları 14

1. Use Case'in Amacı ve Öğrenme Çıktıları

Bu çalışma tek bir "PDF'ye soru soran chatbot" yapmaktan daha kapsamlıdır. Amaç, Fatih'in üç katmanda ilerlemesidir: önce klasik Retrieval-Augmented Generation (RAG) zincirinin her adımını görmesi, sonra taranmış/görsel ağırlıklı belgelerde OCR ve layout-aware parsing eklemesi, son olarak metin ve görsel kanıtları tek bir cevapta birleştiren hibrit bir doküman zekâsı ürünü oluşturmasıdır.



1.1. Temel öğrenme çıktıları

- RAG'ın neden yalnızca "LLM'e PDF vermek" olmadığını; ingestion, chunking, embedding, retrieval ve generation adımlarından oluşan bir sistem olduğunu açıklayabilmek.
- Embedding, cosine similarity, vector database, top-k retrieval, chunk size ve overlap kavramlarını uygulamalı olarak görmek.
- LLM'in cevap üretirken yalnızca getirilen bağlamı kullanmasını sağlayan grounding yaklaşımını ve hallucination riskini anlamak.
- OCR'ın görüntüdeki yazıyı makine tarafından işlenebilir metne dönüştürdüğünü; ancak fotoğraf/grafik anlamlandırmanın OCR'dan farklı bir problem olduğunu kavramak.
- Taranmış PDF, dijital PDF, tablo, grafik, şekil ve karma içerikli dokümanlara farklı işleme yolları uygulamak.
- Kaynak sayfa numarası, alıntılanan chunk, retrieval skoru ve cevap güveni gibi izlenebilirlik (traceability) özellikleri geliştirmek.
- Bir AI projesini sadece "çalışıyor" seviyesinde değil; test edilen, ölçülen, belgelenen ve demo edilebilen bir portföy projesine dönüştürmek.

Portföy hedefi
Final repo; düzgün README, ekran görüntüleri/GIF, mimari diyagram, örnek veri, test senaryoları, ölçüm tablosu, Docker çalıştırma yönergesi ve kısa demo videosu içermelidir. Bu standart proje sonucunu LinkedIn'de "öğrendim" paylaşımından "ürün geliştirdim ve ölçtüm" paylaşımına taşır.

2. RAG ve OCR'ı Doğru Konumlandırma

2.1. RAG nedir?

Retrieval-Augmented Generation, bir dil modelinin cevap üretmeden önce harici bir bilgi kaynağından ilgili parçaları bulup bu parçaları prompt bağlamına eklemesi yaklaşımıdır. PDF senaryosunda belge doğrudan her soruda bütünüyle modele gönderilmez. Önceden ayrıştırılır, anlamlı parçalara bölünür, vektör temsillerine çevrilir ve sorgu geldiğinde en ilgili parçalar geri çağrılır. Böylece maliyet düşer, uzun dokümanlarla çalışmak kolaylaşır ve cevapların belgeye dayandırılması mümkün olur.

2.2. RAG zincirinin ana bileşenleri

Terim	Bu projedeki anlamı

Document ingestion	PDF'nin sisteme alınması ve sayfa/metin/metadata bilgisinin çıkarılması.
Chunking	Uzun metnin, retrieval için uygun boyuttaki parçalara bölünmesi.
Embedding	Metin parçalarının anlamı temsil eden sayısal vektörlere dönüştürülmesi.
Vector store	Embedding'lerin ve metadata'nın saklandığı, benzerlik aramasının yapıldığı katman.
Retrieval	Kullanıcı sorusuna semantik olarak en yakın chunk'ların bulunması.
Generation	Bulunan kanıtların bir LLM'e verilerek belgeye dayalı doğal dil cevabı oluşturulması.
Grounding / citation	Cevabın hangi sayfa ve parçalara dayandığının gösterilmesi.

2.3. OCR nedir - ne değildir?

Optical Character Recognition (OCR), resim veya taranmış doküman içindeki karakterleri algılayarak bunları metne dönüştürür. Örneğin taranmış bir ders notundaki "Dijkstra algoritması" ifadesini çıkarabilir. Ancak bir grafikte kırmızı çizginin neden yükseldiğini, bir devre şemasındaki bileşenlerin işlevini veya bir fotoğraftaki sahnenin anlamını tek başına OCR çözmez.

Kritik ayrım
OCR = görseldeki yazıyı okuma.
VLM / visual reasoning = görselin içeriğini, düzenini, tabloyu, grafiği veya diyagramı yorumlama.
Final projede bu iki kanal birlikte kullanılmalıdır.

3. Aşama 1 - PDF Tabanlı Temel RAG Web Uygulaması

Bu aşamada amaç mümkün olduğunca az "sihirli framework" kullanarak RAG'ın mekanizmasını gözle görünür hale getirmektir. Fatih'in ilk sürümde ingestion, chunking, embedding, retrieval ve prompt oluşturma fonksiyonlarını ayrı modüller olarak yazması önerilir. LangChain veya LlamaIndex daha sonra karşılaştırma amacıyla eklenebilir.

3.1. Use Case

UC-01: Ask My PDF
Kullanıcı web arayüzünden metin tabanlı bir PDF yükler. Sistem belgeyi ayrıştırır, parçalara böler ve embedding'lerini üretir. Kullanıcı belge hakkında soru sorduğunda sistem en ilgili parçaları bulur, yalnızca bu bağlamı LLM'e verir ve sayfa numarasıyla birlikte cevap üretir.

3.2. Aktörler

- Birincil aktör: PDF yükleyen ve soru soran kullanıcı.

- Sistem aktörleri: PDF parser, embedding modeli, vector store, retriever ve LLM.

3.3. Ön koşullar

- Python 3.11+ kurulmuş olmalı.
- Temel Python bilgisi, HTTP/REST mantığı ve Git kullanımı bilinmeli.
- İlk testler için 5-30 sayfalık dijital (taranmamış) PDF kullanılmalı.
- LLM için ya API tabanlı model ya da Ollama üzerinden yerel bir model seçilmeli. API anahtarı repository'e kesinlikle commit edilmemeli.

3.4. Önerilen teknoloji yığını

Katman	Öneri	Neden
Backend	Python + FastAPI	AI/NLP ekosistemi güçlü; dosya upload ve REST API sade.
PDF parsing	PyMuPDF (fitz)	Sayfa metni, bloklar, görseller ve metadata üzerinde güçlü kontrol.
Embedding	sentence-transformers	Ücretsiz/yerel embedding ile RAG mekanizmasını şeffaf öğrenme.
Vector DB - başlangıç	FAISS veya Chroma	Tek makinede hızlı prototip.
Vector DB - gelişmiş	Qdrant	Metadata filtreleme, hybrid search ve üretime yakın deneyim.
LLM	Ollama veya API tabanlı bir LLM	Yerel maliyetsiz deney ya da daha güçlü bulut modeli seçeneği.
Frontend	React/Next.js veya sade HTML+JS	Fatih'in frontend seviyesine göre. Portföy için Next.js daha güçlü.
Test	pytest	Chunking, retrieval ve API davranışlarını otomatik test etmek için.

3.5. Fonksiyonel akış

1. Kullanıcı PDF seçer ve yükler.
2. Backend dosyayı doğrular (MIME type, boyut, sayfa sayısı).
3. PyMuPDF her sayfanın metnini çıkarır; page_number metadata olarak korunur.
4. Metin örneğin 500-800 token civarında chunk'lara ayrılır; %10-20 overlap denenir.
5. Her chunk embedding modelinden geçirilir ve vector store'a yazılır.
6. Soru geldiğinde soru embedding'i hesaplanır ve top-k (ör. 4-6) chunk getirilir.
7. Retriever çıktıları sayfa bilgileriyle prompt içine yerleştirilir.
8. LLM'e "yalnızca verilen kaynaklara dayan; kanıt yoksa bilmiyorum de" kuralı verilir.
9. UI cevabın altında kullanılan sayfa numaralarını ve kısa kaynak parçalarını gösterir.

3.6. İlk aşama için zorunlu özellikler

- Tek PDF yükleme ve işleme durumu göstergesi.
- Belge adı ve toplam sayfa bilgisini gösterme.
- Soru-cevap chat alanı.
- Her cevapta "Kaynaklar: s. 3, s. 7" gibi referans.

- Retrieved chunk'ları geliştirici panelinde gösterebilme.
- "Bu bilgi belgede yok" senaryosunda modelin uydurmamasını test etme.
- Aynı PDF tekrar yüklendiğinde gereksiz yeniden embedding üretimini önlemek için basit hash/cache.

3.7. Aşama 1 acceptance criteria

Kriter	Başarı tanımı
AC1	Metin tabanlı PDF yüklenebiliyor ve sistem hatasız index oluşturuyor.
AC2	Belgede açıkça bulunan en az 20 soruluk test setinde cevapların çoğu doğru kaynağa yönleniyor.
AC3	Her cevap en az bir sayfa/chunk kaynağıyla ilişkilendiriliyor.
AC4	Belgede bulunmayan en az 5 soruda sistem uydurma yerine belirsizlik bildiriyor.
AC5	README'de RAG pipeline diyagramı ve kullanılan chunk/embedding ayarları belgeleniyor.

4. Aşama 2 - OCR ve Multimodal Doküman Anlama

İkinci aşamada aynı uygulama, sadece dijital metin PDF'lerinde değil; taranmış sayfalar, ekran görüntüleri, tablo ve görsel bileşenler içeren dokümanlarda da çalışacak şekilde genişletilir. Burada hedef "OCR ekledim" demekten öte, hangi sayfanın hangi işleme yoluna gideceğini belirleyen bir document processing pipeline geliştirmektir.

4.1. Use Case

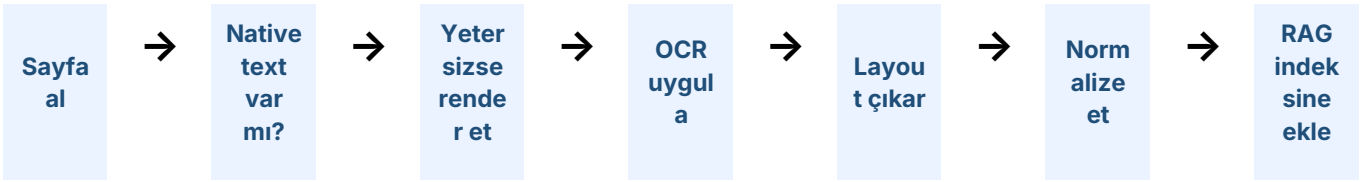
UC-02: Read Scanned & Visual PDFs
Kullanıcı taranmış veya görsel ağırlıklı bir PDF yükler. Sistem metin katmanı zayıf olan sayfaları otomatik belirler; OCR çalıştırır, layout öğelerini çıkarır ve gerekirse sayfanın/görselin semantik açıklamasını bir VLM ile üretir. Bu çıktılar RAG indeksine metadata ile eklenir. Kullanıcı daha sonra metin, tablo, ekran görüntüsü veya grafik hakkında soru sorabilir.

4.2. OCR için öneriler

Önerilen başlangıç motoru PaddleOCR'dır. Güncel sürümleri çok dilli OCR, belge ön işleme ve yapılandırılmış doküman parsing yetenekleri sağlar. Alternatif olarak Docling; PDF, Office ve görsel belgeleri ortak bir doküman temsilinde işlemesi ve farklı OCR back-end'leriyle çalışabilmesi nedeniyle deneysel karşılaştırma için uygundur.

- PaddleOCR: taranmış metin, eğik/bozuk görüntü, çok dil ve document parsing denemeleri için.
- Docling: layout-aware parsing, tablo/başlık/paragraf yapısını korumak ve Markdown/JSON çıkışı karşılaştırmak için.
- PyMuPDF: önce sayfada zaten extract edilebilir metin var mı kontrol etmek ve yalnızca gerektiğinde OCR tetiklemek için.

4.3. OCR pipeline tasarımı



Basit fakat etkili karar kuralı: bir sayfadan çok az metin çıkıyorsa veya metin/glyph kalitesi şüpheliyse sayfayı 200-300 DPI görüntüye render edip OCR'a gönder. Böylece her sayfada gereksiz OCR maliyeti oluşturulmaz.

4.4. Görsel çıkarım katmanı

OCR sonrası iki veri türü ayrılmalıdır: (1) görüntüde bulunan yazı, (2) görüntünün anlamı. İkinci tür için küçük/orta bir Vision-Language Model kullanılabilir. Örneğin bir grafik görseli modele "Bu grafiğin eksenleri nedir, en yüksek nokta hangisidir, ana eğilim nedir?" gibi yapılandırılmış bir promptla açıklattırılır. Üretilen açıklama, image_id ve page_number metadata'sıyla ayrı bir chunk olarak indekslenir.

Neden ayrı chunk?
OCR metni ile VLM açıklamasını tek metinde eritmek yerine modality metadata ile ayrı tutmak, retrieval sırasında "bu kanıt OCR'dan mı, native text'ten mi, görsel yorumdan mı geldi?" sorusunu cevaplamayı kolaylaştırır.

4.5. Önerilen metadata şeması

```
{
  "document_id": "sha256...",
  "page": 7,
  "content_type": "native_text | ocr_text | table | image_caption",
  "source_bbox": [x0, y0, x1, y1],
  "image_id": "page7_img2",
  "ocr_confidence": 0.94,
  "text": "...",
  "embedding": "vector"
}
```

4.6. Aşama 2 için zorunlu test seti

- Taranmış ders notu: native text yok, sadece OCR ile cevaplanabilmeli.
- Metin + tablo içeren rapor: tablo satır/sütun ilişkileri mümkün olduğunca korunmalı.
- Grafik içeren rapor: OCR eksen yazılarını okuyabilmeli; VLM ana eğilimi yorumlayabilmeli.
- Türkçe karakterler (ğ, ş, ı, İ, ö, ü, ç) içeren sayfalar.
- Düşük çözünürlüklü veya hafif eğik tarama.
- Belgede olmayan bir görsel bilgi sorusu: sistem bunu kaynak yok diye reddedebilmeli.

4.7. Acceptance criteria

- Native text bulunmayan PDF'de OCR otomatik devreye giriyor.
- OCR sonucu ile kaynak sayfa arasında ilişki korunuyor.

- Cevap arayüzünde kanıtın türü (Text/OCR/Image/Table) görülebiliyor.
- En az 10 OCR sorusu + 5 görsel/tablo sorusundan oluşan ölçüm seti repository'de tutuluyor.
- OCR motoru değiştirildiğinde aynı testler tekrar çalıştırılıp sonuç tablosu üretilebiliyor.

5. Aşama 3 - Final Projesi: HybridDoc AI

Final proje, ilk iki aşamanın kodunu çöpe atmadan genişleten bir "hibrit PDF okuyucu / document intelligence assistant" olacaktır. Çalışma adı olarak HybridDoc AI kullanılabilir. Amaç yalnızca chat ekranı değil, dokümanın hangi kanıtlara dayanarak cevaplandığını görünür yapan açıklanabilir bir ürün geliştirmektir.

5.1. Ürün vizyonu

HybridDoc AI
Metin tabanlı, taranmış ve görsel ağırlıklı PDF'leri aynı sistemde işleyen; RAG + OCR + görsel retrieval + kaynak doğrulama kullanan; her cevabın sayfa ve içerik türü kanıtlarını gösteren açık kaynak bir multimodal PDF intelligence platformu.

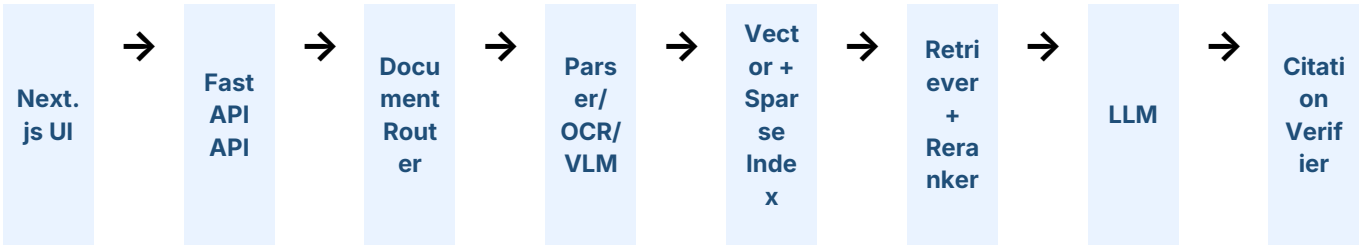
5.2. Final sistemin ayırt edici özellikleri

- Adaptive ingestion: her sayfayı native-text, OCR-required veya visual-heavy olarak sınıflandır.
- Hybrid retrieval: dense semantic retrieval + isteğe bağlı BM25/sparse retrieval sonuçlarını birleştir.
- Reranking: ilk retrieval adaylarını cross-encoder/reranker ile tekrar sırala.
- Multimodal evidence: metin chunk'ları yanında sayfa görseli veya image-caption kanıtları getir.
- Citations: cevabın yanında sayfa numarası, kanıt snippet'i ve content_type göster.
- PDF viewer sync: kaynak kartına tıklayınca ilgili sayfaya git ve mümkünse bounding box'ı highlight et.
- No-evidence policy: retrieval skoru/evidence yetersizse cevap yerine "belgede doğrulanabilir kanıt bulamadım" de.
- Document comparison mode: iki PDF yükleyip aynı konu için farklılıkları kaynaklarla karşılaştır.
- Export: cevap ve kaynakları Markdown/PDF raporu olarak dışa aktar.
- Evaluation dashboard: retrieval hit-rate, latency, OCR confidence ve cevap doğruluk sonuçlarını göster.

5.3. Gelişmiş - isteğe bağlı "WOW" özellikleri

- Visual document retrieval: sayfaları görüntü olarak embed eden ColPali/ColQwen benzeri modelleri ayrı bir deney kolunda değerlendirme.
- Question routing: "metin sorusu / tablo sorusu / görsel sorusu" şeklinde sorgu sınıflandırıp farklı retriever kullanma.
- Answer verification: oluşturulan cevabın her iddiasını retrieved evidence ile ikinci bir doğrulama adımından geçirme.
- Query expansion: kullanıcı sorusundan anahtar terim ve olası eş anlamlıları üretip hybrid search kalitesini artırma.
- Multi-document workspace: bir klasördeki birden fazla PDF üzerinde ortak arama ve belge filtresi.
- Local-first mode: embedding, vector DB ve küçük LLM'i tamamen yerelde çalıştırabilme.

5.4. Sistem mimarisi



Mimari iki ana yola ayrılır: ingestion pipeline belge yüklendiğinde bir kez çalışır; query pipeline ise her soru geldiğinde çalışır. Bu ayrım hem performansı hem de kod organizasyonunu öğretmek açısından önemlidir.

5.5. Ingestion pipeline

10. Dosya doğrulama ve SHA-256 document_id üretme.
11. Sayfaları tek tek analiz etme: native text yoğunluğu, görüntü sayısı, tarama olasılığı.
12. Native text extraction.
13. Gereken sayfalarda OCR.
14. Tablo/layout parsing.
15. Önemli görseller için VLM caption/structured description.
16. Her veri öğesine page, bbox, modality, confidence metadata ekleme.
17. Chunking + embedding.
18. Dense index ve gerekiyorsa sparse/BM25 index oluşturma.
19. Belge işlem özetini veritabanında tutma.

5.6. Query pipeline

20. Soru normalizasyonu ve gerekiyorsa query routing.
21. Dense search + sparse search.
22. Sonuç birleştirme ve reranking.
23. Kanıtların page/modality bilgisiyle context oluşturulması.
24. LLM generation: yalnızca kanıta dayalı cevap.
25. Citation verifier: cevaptaki ana iddiaların en az bir retrieved evidence ile eşleşmesini kontrol etme.
26. UI: cevap + kaynak kartları + sayfa önizlemesi.

5.7. Final proje için örnek kullanıcı hikâyeleri

- Bir öğrenci olarak 120 sayfalık ders kitabımı yükleyip “Bellman-Ford ile Dijkstra arasındaki temel fark nedir?” diye sormak ve cevabın hangi sayfalardan geldiğini görmek istiyorum.
- Bir araştırmacı olarak taranmış eski bir makaledeki tabloyu sorgulamak ve OCR ile çıkarılan hücrelere dayalı cevap almak istiyorum.
- Bir mühendis olarak rapordaki grafikte en yüksek sıcaklığın hangi testte oluştuğunu sorup görsel kanıtı görmek istiyorum.
- Bir kullanıcı olarak cevabın kaynağına tıklayıp PDF’de doğrudan ilgili sayfaya gitmek istiyorum.
- Bir geliştirici olarak hangi retrieval ayarının daha iyi olduğunu ölçmek için evaluation dashboard üzerinden deney sonuçlarını karşılaştırmak istiyorum.

6. Önerilen Teknik Mimari ve Teknoloji Yığını

6.1. “Öğrenme” ve “portföy” stack’ini ayır

Fatih’in ilk günden çok fazla framework kullanması RAG’ı öğrenmesini zorlaştırabilir. Bu yüzden Aşama 1’de çekirdeği mümkün olduğunca açık yazması; Aşama 2-3’te LangChain/LlamaIndex/Docling gibi abstraction katmanlarını karşılaştırması önerilir.

Bileşen	Öneri	Gerekçe
Dil	Python 3.11+	RAG/OCR/ML ekosistemi; FastAPI ile doğal uyum.
Backend	FastAPI	Async API, dosya upload, otomatik OpenAPI dokümantasyonu.
Frontend	Next.js + TypeScript	Portföy kalitesi, component tabanlı UI ve ürün görünümü.
PDF	PyMuPDF / PyMuPDF4LLM	Metin, blok, sayfa, resim ve Markdown akışları.
OCR	PaddleOCR	Güncel PP-OCR ailesi ve document parsing araçları.
Structured parsing	Docling veya PP-Structure	Layout, tablo, başlık ve belge hiyerarşisi.
Embedding	Sentence Transformers	Yerel, tekrarlanabilir semantic search.
Vector DB	Qdrant	Dense/sparse/hybrid ve metadata tabanlı retrieval.
Dev vector DB	FAISS/Chroma	İlk prototipte düşük kurulum yükü.
LLM orchestration	İlk sürüm manuel; sonra LangChain/LlamaIndex	Önce temeli öğren, sonra framework karşılaştır.
VLM	Qwen3-VL küçük/orta model veya API VLM	Grafik/şekil/screenshot semantiği için; OCR yerine değil OCR yanında.
Cache/queue	Redis (opsiyonel)	Uzun PDF işleme ve tekrar eden sorgular için.
DB	SQLite → PostgreSQL	Belge metadata, kullanıcı ve evaluation sonuçları.
Deployment	Docker Compose	Backend + Qdrant + frontend’i tekrarlanabilir çalıştırma.
Testing	pytest + küçük golden dataset	Regresyon ve retrieval ölçümü.

6.2. Önerilen repository yapısı

```
hybriddoc-ai/
├─ apps/
│  ├─ web/                # Next.js UI
│  └─ api/                # FastAPI
├─ src/
```

```
│   ├─ ingestion/           # parse, OCR, layout, image extraction
│   ├─ chunking/
│   ├─ embeddings/
│   ├─ retrieval/           # dense, sparse, rerank
│   ├─ generation/          # prompt + LLM
│   ├─ citations/
│   └─ evaluation/
├─ tests/
├─ eval_data/
├─ sample_docs/
├─ docs/                    # architecture + screenshots
├─ docker-compose.yml
├─ .env.example
└─ README.md
```

6.3. Başlangıç için önerilen model seçimi

Model isimlerini projenin merkezine koymamak daha doğrudur; çünkü model ekosistemi hızlı değişir. Kodda provider interface kullanılması önerilir: EmbeddingProvider, LLMProvider ve VisionProvider. Böylece yerel model, Hugging Face modeli veya API arasında tek bir konfigürasyon değişikliğiyle geçilebilir.

Mühendislik ilkesi
"Modeli ürüne gömme; modeli bir provider olarak soyutla." Bu tasarım Fatih'e dependency inversion ve değiştirilebilir AI bileşenleri fikrini de öğretir.

7. Test, Ölçüm ve Başarı Kriterleri

RAG projelerinde en sık yapılan hata, birkaç sorunun doğru cevaplanmasını başarı saymaktır. Final repository'de küçük ama kontrollü bir evaluation dataset bulunmalıdır.

7.1. Golden test set

- En az 5 farklı PDF türü: dijital metin, taranmış, tablo yoğun, grafik yoğun, karma.
- Her belge için 10-20 soru ve beklenen sayfa/kanıt etiketi.
- Soru türleri: fact lookup, multi-chunk reasoning, table, visual, not-in-document.
- Her soruda "beklenen cevap" yanında "beklenen evidence pages" tutulmalı.

7.2. Ölçülecek metrikler

Metrik	Ne ölçer?
Retrieval Hit@k	Beklenen kanıt sayfası top-k sonuçlar içinde mi?
MRR	Doğru kanıt retrieval sıralamasında ne kadar üstte?
Answer correctness	Cevabın ground-truth ile anlamsal doğruluğu.
Faithfulness / groundedness	Cevaptaki iddialar retrieved evidence ile destekleniyor mu?
Citation accuracy	Gösterilen sayfa gerçekten cevabı destekliyor mu?
OCR quality	Karakter/kelime düzeyinde örnek doğruluk veya manuel hata oranı.

Latency	Upload işleme süresi ve soru başına cevap süresi.
Cost	API kullanılıyorsa belge/soru başına yaklaşık maliyet.

7.3. Deneyler

- Chunk size: 300 / 600 / 900 token karşılaştır.
- Overlap: 0 / %10 / %20 karşılaştır.
- Top-k: 3 / 5 / 8 karşılaştır.
- Dense-only vs hybrid retrieval karşılaştır.
- Reranker kapalı/açık karşılaştır.
- Native extraction vs OCR output kalitesini aynı sayfada karşılaştır.
- Text-RAG vs visual retrieval yaklaşımını grafik/tablo sorularında karşılaştır.

GitHub'da fark yaratacak kısım
README'de "Best configuration" demek yerine deney tablosu koy: ayar, Hit@5, citation accuracy, latency. Bu tek başına projeyi sıradan tutorial repo'larından ayırır.

8. GitHub ve LinkedIn Portföy Standardı

8.1. README yapısı

- 30 saniyede anlaşılır proje özeti.
- Demo GIF/video bağlantısı.
- Architecture diagram.
- Features listesi.
- RAG/OCR/VLM pipeline açıklaması.
- Quick start (Docker Compose önerilir).
- .env.example ve model/provider seçenekleri.
- Evaluation sonuç tablosu.
- Known limitations.
- Roadmap.
- Kaynaklar ve kullanılan açık kaynak projelere atıf.

8.2. GitHub commit/milestone önerisi

- v0.1 - Native PDF parser + chunking
- v0.2 - Embeddings + vector search
- v0.3 - RAG chat + citations
- v0.4 - OCR fallback
- v0.5 - Layout/table extraction
- v0.6 - Vision evidence
- v0.7 - Hybrid retrieval + reranker
- v1.0 - Evaluation + Docker + polished UI

8.3. LinkedIn paylaşımında ne anlatılmalı?

Paylaşım "RAG öğrendim" şeklinde değil, problemi ve mühendislik kararlarını öne çıkaracak şekilde yapılmalıdır. Örneğin: dijital PDF ile taranmış PDF'nin aynı parser ile güvenilir çalışmadığını fark ettim;

bunun için adaptive ingestion geliştirdim. OCR'ın görsel semantiği çözmediğini gördüğüm için VLM evidence channel ekledim. Ardından retrieval kalitesini Hit@5 ve citation accuracy ile ölçtüm.

- 1 adet kısa demo videosu/GIF.
- 1 adet mimari diyagram.
- 1 adet evaluation tablosu veya "önce/sonra" retrieval sonucu.
- Teknik etiketler: #RAG #OCR #DocumentAI #MultimodalAI #FastAPI #NextJS #Qdrant #OpenSource.

9. Geliştirme Takvimi ve Teslimat Kontrol Listesi

9.1. 8 haftalık önerilen plan

Zaman	Hedef
1. Hafta	RAG teorisi, PyMuPDF ile extraction, chunking deneyleri.
2. Hafta	Embedding + FAISS/Chroma, semantic retrieval CLI.
3. Hafta	FastAPI + web UI + LLM generation + citations. Aşama 1 tamam.
4. Hafta	OCR teorisi, PaddleOCR entegrasyonu, scanned PDF fallback.
5. Hafta	Layout/table parsing + visual caption/VLM denemeleri. Aşama 2 tamam.
6. Hafta	Qdrant, hybrid search, reranking, multi-document yapı.
7. Hafta	Evaluation dataset, metrikler, latency/cost ölçümü.
8. Hafta	UI polish, Docker, README, demo video, v1.0 release ve LinkedIn paylaşımı.

9.2. Definition of Done - Aşama 1

- [ ] PDF upload çalışıyor
- [ ] Chunk + embedding pipeline modüler
- [ ] Vector search sonuçları incelenebilir
- [ ] LLM cevapları kaynak sayfa gösteriyor
- [ ] Not-in-document testi var
- [ ] README ve ekran görüntüsü var

9.3. Definition of Done - Aşama 2

- [ ] Scanned PDF otomatik algılanıyor
- [ ] OCR fallback çalışıyor
- [ ] Türkçe OCR testi var
- [ ] Table/image metadata korunuyor
- [ ] En az bir görsel yorumlama yolu var
- [ ] Text/OCR/Image kaynak türleri UI'da görülebiliyor

9.4. Definition of Done - Final

- [ ] Hybrid retrieval veya reranking uygulanmış
- [ ] Golden evaluation dataset repository'de
- [ ] Retrieval ve citation metrikleri raporlanmış
- [ ] Docker ile tek komut kurulum
- [ ] UI'da kaynak sayfaya geçiş
- [ ] Known limitations yazılmış
- [ ] v1.0 GitHub release hazırlanmış
- [ ] Demo video/GIF ve LinkedIn paylaşımı hazır

10. Kaynaklar ve Güncel Teknoloji Notları

Aşağıdaki kaynaklar Ekim 2026 itibarıyla bu use case'in teknoloji önerilerini destekleyen güncel resmi veya birincil dokümantasyonlardır. Sürümler zamanla değişebileceği için uygulama sırasında her kütüphanenin güncel "stable" dokümantasyonu kontrol edilmelidir.

[1] PyMuPDF Documentation - PDF metni, blokları ve görselleri çıkarmak; RAG/LangChain/LlamaIndex entegrasyon örnekleri.
https://pymupdf.readthedocs.io/

[2] PaddleOCR Documentation - OCR, PP-OCR ailesi, document parsing ve yapılandırılmış doküman işleme.
https://www.paddleocr.ai/

[3] Docling Documentation - PDF/Office/image belgelerini ortak representation'a dönüştürme ve OCR back-end'leri.
https://docling-project.github.io/docling/

[4] Sentence Transformers - Embedding, semantic search ve 2026'da görsel multi-vector retrieval desteği.
https://www.sbert.net/

[5] Qdrant Documentation - Vector DB, dense/sparse/hybrid retrieval, reranking ve multivector document retrieval örnekleri.
https://qdrant.tech/documentation/

[6] FastAPI Documentation - Dosya upload ve Python API geliştirme.
https://fastapi.tiangolo.com/

[7] Next.js Documentation - Web arayüzü ve App Router tabanlı full-stack React geliştirme.
https://nextjs.org/docs

[8] Hugging Face - Visual Document Retrieval - ColPali tarzı sayfa-görsel retrieval yaklaşımının güncel örnekleri.
https://huggingface.co/docs/transformers/

[9] Qwen3-VL / Transformers - Görsel-metin anlama için açık kaynak VLM ailesi; final projede opsiyonel provider örneği.
https://huggingface.co/docs/transformers/main/model_doc/qwen3_vl

Son öneri
Fatih Aşama 1'de LangChain/LlamaIndex kullanmadan küçük bir RAG çekirdeği yazarsa, Aşama 2 ve 3'te bu framework'lerin neyi kolaylaştırdığını çok daha iyi anlayacaktır. Projenin asıl değeri seçilen LLM'den değil; doküman ingestion kalitesinden, retrieval tasarımından, kanıt gösteriminden

ve ölçülebilir evaluation sürecinden gelecektir.

— Use Case Sonu —